

06-实施指南-小项目S1-S5

小项目实施指南：面试弹药库（v3 重写版，S1-S6）

2026-10-05 v3 重写（导师与学员讨论定稿）：P1/P2 v3 化后小项目重新分工。配套 03-项目结构定稿.md（v3）§ 3。**定位铁律：小项目不写简历**，用途是面试弹药 + 大项目扫雷先导。评价标准只有一条——**单位时间产出的面试弹药密度**。

0. 通用要求

- 1. 独立仓库 + README（设计取舍 + 实验数据 + 复现命令）；代码是给自己的，**README 和实验记录才是弹药**；
- 2. 每个项目必须攒齐”弹药三件套”：**设计取舍、踩过的一个坑、一张数据/对照表**；
- 3. **硬时间盒**：每个项目开工前在 README 顶部写下截止日期；超期砍功能不砍质量，不许”再打磨一周”；
- 4. 弹药纪律：做完立刻写一页《复盘卡》（如果重来会怎么改 / 被追问时的三个薄弱点），面试前只看复盘卡；
- 5. 名额纪律：独立占工期的小项目 **≤3 个**——S1 必做，S6（平台岗）或 S4（存储岗）二选一，S3 有余力再做；S2 并入 P1 不单列，S5 默认不做；
- 6. **微弹药 M1-M3 另立体系**（见后文）：按天计的练手小品，不是项目、不写简历、不占名额，总投入 ≤1 周。

#	项目	v3 定位	时间盒
S1	mini-Raft KV（Go）	必做	3-4 周硬顶
S6	K8s 调度插件实战（Go）	平台/AI Infra 岗首选，新增	3-4 周（阶段 C 与 P1 步骤 7 合流，净 2-3 周）
S4	bitcask mini 存储引擎（Go）	存储岗首选，与 S6 二选一	3-4 周
S3	eBPF 观测工具（Go + bpf2go）	可选，P1 核心 1 完工后	1-2 周
S2	C++/io_uring 预取器	并入 P1（04 § 9.2 + A 轨实验臂）	随 P1
S5	mini-RPC / 自研网络框架	默认不做（仅网络中间件岗改形态保留）	—

S1 mini-Raft KV（必做，3-4 周硬顶）

介绍：Go 实现 3 节点 Raft，暴露 Get/Put。目的：把一致性从”背”变”讲”。**清醒认知：**这是 MIT 6.824 人手一个的项目，**实现本身差异化≈零，简历上永不排第一**；弹药价值在工程细节、实验设计与工业对读。

范围：选举、日志复制、快照、崩溃恢复；**不做：**membership 变更、Pre-Vote（列为”必须能白板讲”）。

关键设计（保留 v2）

- 状态机三角色 + 单 tick 驱动定时器（`time.Timer` 随机化 $[T, 2T)$ ， $T \approx 500\text{ms}$ ）；
- **必须持久化三字段：**`currentTerm`、`votedFor`、`log[]`——经典死亡陷阱：`votedFor` 不落盘 → 同任期投两票 → 脑裂。WAL：`len` | `crc32` | `entry`，追加写；
- RPC（gRPC）`RequestVote` / `AppendEntries`：**每个回复先比 term，小则让位**——第二大陷阱；
- 提交规则：leader 只提交**当前任期**日志（§ 5.4.2）；`prevLogIndex`/`term` 不匹配就回退 `nextIndex`，冲突项加速回退是加分项；
- 线性一致读（stretch）：`ReadIndex` 或走日志；要能讲”为什么 `ReadIndex` 比 `lease` 对时钟依赖小”；
- 快照：`lastIncludedIndex` 前截断 + `InstallSnapshot`；恢复时快照 + 后续日志重放。

v3 增强（三个动作把弹药从 60 分提到 85 分）

1. **工业对读（必做）：**toy 跑通后精读 etcd raft 库——`node.go` 的 Ready 循环、`unstable log`、`Storage` 接口、`ReadIndex` 的工业实现。产出一篇《我的 toy raft 与 etcd raft 的 5 个差异》（候选：批量化 Ready 通道、消息流水线、leader 转移、检查点时机、无环日志结构）。“写过且读过工业实现”和”做过 Lab”在面试中是两个物种；
2. **实验一换血：删 30% 丢包单点**（太极端，只能得到”它瘫了”），改为**丢包率扫描 0.1% / 1% / 5% / 10%**，画 commit P50/P99 随丢包率曲线，并解释拐点（重传与选举超时的耦合）；
3. **实验二新增：无主停机窗口分布**——`kill -9 leader ≥ 100` 次，记录从 kill 到新 leader 首个成功写的时间分布（中位数/P99）。这是比提交延迟更受分布式面试关注的**可用性**数字。分区实验区分三种：多数派隔离、少数派隔离、不对称分区（leader 单向失联），分别给行为表。

实施步骤

1. 单机 KV + WAL；2. gRPC 骨架 + 选举（打印每次 term/投票流向）；3. 复制 + 提交（`raftd` 集成 harness：延迟/分区/重启注入）；4. 快照 + `kill -9` 恢复；5. v3 两组实验 + etcd 对读笔记。

验收：故障注入下已提交数据零丢失；三类分区行为表；丢包扫描曲线；无主窗口分布（≥100 样本）；etcd 差异笔记。**能讲的点：**`votedFor` 持久化；只提交当前任期日志；`ReadIndex` vs `Lease`；**我的 toy 与 etcd 的具体差异**；Pre-Vote 防分区节点旧 term 扰乱；joint consensus 两阶段成员变更（不实现但白板讲）。

S2 C++/io_uring 预取器（v3：不再单列，并入 P1）

定位变化：P1 v3 § 9.2 已收编为 A 轨（小文件高 IOPS 臂）的实验分支。独立仓库可以，但**不占小项目名额、不另排独立工期**，随 P1 步骤 8 前后推进。

范围：本地写路径 + 提交模型；**不做：**S3 完整客户端（HTTP Range 够用）。

关键设计（保留 v2）：

- `io_uring` (liburing)：队列深度 64–256 可配；`IORING_REGISTER_FILES` + registered buffer 消 per-op 开销；
- **O_DIRECT 三重对齐坑**：缓冲区地址、长度、偏移都对齐逻辑块（`posix_memalign` 4K）；
- **overlayfs 坑**：容器内 overlayfs 上 O_DIRECT 可能 EINVAL——目标路径必须落宿主机真实 ext4/xfs（hostPath 或宿主机直接跑）；
- 提交/收割：`get_sqe` → `prep` → `submit_and_wait(N)`，CQE 批量收割 + `resubmit`，错误码逐个处理（短读=EOF）；
- **fio 校准**：先测裸盘直写上限贴着比，“比 fio 差 8%”比“我很快”专业十倍。

v3 补强：

- 步骤 0 先 `uname -r` 锁内核版本（registered files/buffers 行为随版本变），并**逐个通关 liburing 官方 examples**——`io_uring` + O_DIRECT + overlayfs 三坑叠加直接上手太陡，C++ 是次语言，坡度要削；
- **收益预期预先诚实化**：P1 主臂 4MiB 块经 HTTP 抓取是**网络 bound**，本地写路径省 syscall 对吞吐贡献预期≈0；`io_uring` 的战场是 A 轨小文件高 IOPS。报告正确姿势：“syscall 数大降、大块吞吐不变（并解释为何网络 bound）、小文件臂 IOPS ↑X”，不许只挑好看的数字。

实施步骤：1. Go 线程池+buffered 基线 → 2. C++ buffered（隔离语言因素）→ 3. O_DIRECT+`io_uring` → 4. registered 优化版 → 5. 参数矩阵（QD × 块大小 4K/64K/1M × 并发，吞吐 + `strace` syscall 数 + CPU）。**弹药：**O_DIRECT 对齐与 overlayfs 坑；`io_uring` 减少了什么（syscall + 线程调度）；**为什么大块场景它不赚**——能讲清”不适用边界”比吹收益更值钱。

S3 eBPF 观测工具（v3：可选，后置，MVP 砍到一个探针）

定位变化：原排期”P1 指标期”与核心 1 旗舰图争抢时间，后移到 **P1 核心 1 完工后**有余力再做；MVP 只做一个探针，不做全家桶。

范围：只读观测；内核 ≥5.10；宿主机跑（容器权限麻烦，先宿主机）。

v3 范围：

- **MVP（唯一必做）：**block I/O 延迟直方图——`tracepoint block/block_rq_issue` + `block/block_rq_complete` 按 request 指针配对，内核侧 log2 bucket 聚合，用户态周期拉 per-CPU map；
- 第二（可选）：`major fault`（`exceptions/page_fault_user` 的 major/minor 位）；第三（可选）：`sys_enter_read` 字节量；
- **必须写进报告的观测逻辑：**P1 preheat 模型下缓存命中 page cache 时**根本不产生 block I/O**——block I/O 直方图天然区分命中/未命中；但 **readahead 陷阱**：内核预读造成的 major fault 不记在消费者进程名下，用进程维度归因会算错。这个陷阱本身就是上等弹药；
- CO-RE：`vmlinux.h` + `bpf2go`；优先 `tracepoint` 少用 `kprobe`（ctx 字段跨版本稳）；降级方案 `/proc/diskstats` 采样，诚实标注口径差异。

实施步骤：1. bpf2go hello-world 验环境 → 2. block I/O 配对延迟直方图 + Go 聚合 → 3. (可选) fault/read → 4. exporter 接入 P1 实验，与 dataloader 打点交叉验证口径。**验收：**P1 实验期间稳定出数；两口径交叉验证一致；readahead 归因陷阱有专门一节。**弹药：**per-CPU map vs ringbuf 取舍；page cache 命中为何无 block I/O；readahead 如何污染进程归因；CO-RE 解决什么。

S4 bitcask mini 存储引擎（存储岗首选，3-4 周）

介绍：Go 实现 bitcask：WAL + 内存索引 + hint file + merge。**范围：**单机；**不做：**分布式（S1 覆盖）、全量 LSM（对读即可）。

关键设计（保留 v2）：

- 记录格式：`crc32 | tstamp | keysz(4) | valsz(4) | key | value`，tombstone 用 valsz 特殊标记（删除也是写）；
- 内存索引：`map[key] → {fileID, offset, size}`；**1 亿 key 的内存账要在自己机器上实测给数字**（key 均长 × key 数 × Go map 开销）——这是 bitcask 的适用边界；
- 活跃文件写满（256MB）转 immutable；**fsync 三档**做成配置：逐条 / group commit（N ms） / 只靠 OS ——benchmark 三档对比，存储引擎最核心权衡之一；
- merge：immutable 活记录 → 新数据文件 + hint file → **原子切换**：先 rename 数据/hint，最后写 manifest 指向新文件集；把每个崩溃窗口想清楚写进注释；
- 读路径：索引命中 → pread → crc 校验（失败=盘上腐坏，报错或触发重 merge）。

v3 四处补强

1. **时间戳改 8 字节**：4 字节秒级时间戳有 2038 问题，记录头用 int64 毫秒/微秒；主动在注释里写出这个修正——小细节，大印象；
2. **benchmark 提到 10M key 档**（32G 内存机器跑得动）：1M 算不出 bitcask 的命门——全内存索引的容量账，必须在 10M 档实测启动时间、常驻内存、读写吞吐，画出“索引内存/数据量”比；
3. **明确做单写多读模型并讲透**：写锁天然串行化（bitcask/LSM 共同核心约束），读无锁；压测多写者排队与多读扩展性；
4. **加一次 LSM 对读**（不实现）：读 LevelDB/RocksDB compaction 文档，终点是白板回答“数据量超内存后 bitcask 为什么输给 LSM（SSTable + 稀疏索引 + 分层 compaction 的读写放大对比）”。

实施步骤：1. append-only log + 读路径 → 2. 多文件轮转 + tombstone → 3. 启动扫描（暴力→hint file，对比启动时间）→ 4. merge + manifest 原子切换 → 5. fuzz 随机点 kill -9 崩溃矩阵 → 6. benchmark：1M/10M 两档、merge 期间写延迟抖动（**丑数字敢贴**）、写放大计算、fsync 三档。**验收：**崩溃矩阵全绿；10M 档容量账实测；merge 抖动数据；LSM 对读笔记一页。**弹药：**fsync 三档；merge 原子切换崩溃窗口；单写多读约束；10M 实测内存账；bitcask vs LSM 适用边界。

S5 mini-RPC / 自研网络框架（v3：默认不做）

砍掉的理由（想恢复前重读）：

1. 自研 RPC 是 2026 年校招最过剩品类，文档旧版自己都要靠”io_uring/zero-copy 红线”壮胆；
2. P2 v3 明确不重造数据面，它对大项目的反哺链已断；
3. MSG_ZEROCOPY 只在超大 payload 有收益，RPC 场景极可能做出不好看的阴性数字——**小项目的阴性结果没有大项目那种测量纪律背书，不值钱。**

唯一恢复条件：投递目标明确是网络中间件/服务网格数据面岗。且形态改为：**grpc-go 连接模式 + HTTP/2 流控的 benchmark 与火焰图剖析**（多连接 vs 单连接多路复用、流控窗口对 P99 的影响、keepalive 与断连重建），外加一个”优化 PR 级尝试”（如自定义 balancer/Msg 批处理）；**不再造私有协议**。时间盒 2 周，对照 grpc-go 实测。

S6 K8s 调度插件实战：拓扑感知打分与双目标冲突（新增，平台/AI Infra 岗首选）

介绍：基于 kubernetes-sigs/scheduler-plugins，跑通官方拓扑感知调度 → 自研最小 PreFilter+Score 插件走通全链路 → 在”数据亲和 × 拓扑亲和”冲突点做增量实验。**为什么是它：**所有小项目中与 P1 咬合最紧——P1 核心 1 就是 Gate + Score plugin，本项目等于给 P1 步骤 7 提前扫雷；NUMA/设备拓扑是 AI Infra 平台岗面试高频题；scheduler-plugins 是官方仓库，可顺带练习开源流程（**但不背开源名额**，主回馈仍是 Fluid/AIBrix）。

事实前提（2026-10 核实，决定姿势）

- 官方已有 NodeResourceTopologyMatch (Filter) + NodeResourceTopologyResourceAllocationScore (Score)，消费 NodeResourceTopology CRD (KEP-119)，由 NFD (Node Feature Discovery) + topology-updater 上报；
- Topology Manager 自 k8s 1.27 GA；
- **DRA 已不是”未来趋势”而是当下事实：**Dynamic Resource Allocation 自 **Kubernetes v1.35 GA** (stable，特性门控锁定不可关闭，v1.30 首次引入)；v1.36 (2026-05) 进一步把 ResourceClaim 接入新的 Workload/PodGroup (内核 gang scheduling + topology-aware scheduling) 体系。但 2026 年的现网现实是：NRT 插件仍是主流、各厂商 DRA driver 生态在迁移期——**因此 S6 的正确叙事是”旧设备模型的亲手实现 + 讲清新世界如何取代它”，两者都要硬** (DRA 白板要点见下)；
- **姿势与大项目同构：复现官方 → 读懂 → 写最小件走通链路 → 冲突点做增量。**绝不声称”发明了拓扑调度器”，官方 NRT 插件就是对照组。

DRA 白板要点（不写代码，阶段 A 同期半页笔记，2026-10 核实）

必须能脱稿讲清四条：

1. **旧 device plugin 模型的痛：**设备被抽象成整数资源 (nvidia.com/gpu: 1)，无法按显存/型号/互联拓扑做结构化请求，不支持设备共享、不支持属性表达式过滤，拓扑信息只能走 Topology Manager + NFD + NRT 这条外挂链路；
2. **DRA 四个核心对象** (resource.k8s.io/v1)：驱动用 **ResourceSlice** 上报设备能力与拓扑，集群管理员定义 **DeviceClass** (类比 StorageClass)，工作负载用 **ResourceClaim / ResourceClaimTemplate** 申领

（类比 PVC/PV 的申领体验），调度器据此做结构化匹配、kubelet 与驱动完成分配握手；支持 CEL 属性过滤与设备共享；

3. **与 S6/NRT 的关系**：DRA 让设备拓扑成为调度器可直接消费的结构化属性，长期会收编 NFD+NRT 外挂链路的一部分；但迁移期两套并存，NRT 插件在现网仍被广泛部署——能讲清”哪部分被取代、哪部分（NUMA/CPU 亲和）仍独立存在、迁移卡在哪里（驱动成熟度/存量集群版本）“才是满分答案；
4. **前沿联动**：v1.36 Workload/PodGroup API 在内核做 gang scheduling + topology-aware scheduling 并接入 DRA——这与 P1 的训练 Job 调度、S6 的拓扑亲和是同一演进方向，面试可作为”我在跟踪内核调度演进”的谈资，但明确不跟进实现。

两小时手感练习（可选，不过夜、不写简历、不挤占 P1）：阶段 A 完成后，用 kind 建 ≥ 1.35 集群，跑官方 `dra-example-driver`（无真实硬件的假设备驱动），创建一个 DeviceClass + ResourceClaim，`kubectl describe resourceclaim` 看到分配结果与 ResourceSlice 内容即收工。目标只是”见过真的 ResourceSlice/ResourceClaim 长什么样”。

硬件诚实清单（开工第一件事）

- `numactl -H` 验证：游戏本消费级 CPU 几乎肯定是**单 NUMA 节点**、无多 GPU；
- 因此全部拓扑用 fake/annotation 驱动开发——**这完全正当**，官方插件自己的 e2e 测试也 mock NRT 对象；
- 真机多 GPU 拓扑只在 P2 租云真值窗口顺带读取一次（`nvidia-smi topo -m`），不为此单独开预算。

阶段 A：现勘跑通官方链路（约 1 周）

1. kind 集群（版本对齐 scheduler-plugins 兼容矩阵），按仓库 Developer Guide 本地构建镜像（kind load）或用发布 chart；
2. 部署 NFD + topology-updater，生成真实 `NodeResourceTopology` CR；单 NUMA 机器上手写 fake NRT yaml（双 NUMA、fake GPU 资源池）作为第二数据源；
3. 配置 `KubeSchedulerConfiguration` **v1**： `profiles`（schedulerName）、 `plugins`（filter/score 启用 NRT 插件）、 `pluginConfig`（scoringStrategy：MostAllocated/BalancedAllocation/LeastAllocated 各跑一遍看决策差异）；
4. 部署若干请求 CPU/GPU 的测试 Pod，验证绑核/拓扑决策符合预期，抓调度日志；
5. 精读：官方 Score 插件源码（分数如何从 NRT 的 zones/资源分配算出、归一化）、NRT CRD 字段语义；
6. **验收（能口述全链路）**：device plugin `TopologyInfo` → kubelet Topology Manager（与 CPU Manager 协同）→ NFD topology-updater → NRT CRD → 调度插件 Filter/Score → bind。
7. **风险保护**：一周内官方全家桶部署不成功（NFD 组件多、镜像拉取坑），S6 立即降级为”手写 NRT yaml + 只部署插件本体”，再不行降级为”纯跟读源码 + 决策日志推演笔记”，不允许环境黑洞吃掉第二周。

阶段 B：自研最小插件（1-2 周）

1. 插件骨架：`New(obj runtime.Object, handle framework.Handle) (framework.Plugin, error) + PreFilter + Score 两个扩展点`（NormalizeScore 视需要）；
2. 配置：`New` 中读 `pluginConfig` 参数（权重、annotation key），保存到插件状态；

3. **拓扑数据源刻意简化**：先读 node annotation（如 `topology.io/numa-zones: ...` 写 fake 拓扑 JSON），不依赖 NFD 全家桶——目的是走通链路，不是重造上报；
4. 打分逻辑：根据 Pod 请求资源与节点 zone 拓扑，计算”单 NUMA 内可满足度”分；PreFilter 排除明显不可行节点；
5. 工程全套：自定义 scheduler 的 Deployment + ServiceAccount/RBAC（系调度器权限清单本身是弹药）+ ConfigMap 挂载 KubeSchedulerConfiguration；
6. Profile 配置：YAML 定义 profile 与插件启用顺序；本地调试按 Developer Guide（本地跑 scheduler 进程连 kind apiserver / 容器化二选一）；
7. 质量：插件单测（构造 nodes/pods table tests）+ e2e（调度结果符合 annotation 预期）+ 每个调度决策打结构化日志。
8. **验收**：一个独立 `schedulerName` 上”配置 → 编译 → 部署 → 调度生效”全链路走通，单测 e2e 全绿。

阶段 C：双目标冲突实验（1-2 周，与 P1 步骤 7 合流，不额外占工期）

1. 两个 Score 维度并存：**数据亲和分**（P1 的 Job 子集 readyRatio，由 P1 informer 缓存提供）× **拓扑亲和分**（fake NUMA/GPU zone 匹配度）；
2. 构造冲突场景：Job 的热分片缓存在 NUMA-A 节点，空闲 GPU 在 NUMA-A'（拓扑上靠近远端内存）/ 另一节点 NUMA 局部性好但数据冷；
3. 权重 `w` 从 0→1 扫描，画**权重-调度决策曲线**与对应的 makespan/拓扑惩罚代理指标；
4. 终点问题写成报告：框架级多插件分数如何归一化（各插件 Score 值域）、冲突时为什么没有银弹、生产上该让谁暴露权重给用户；
5. **合流动作**：该插件的 Score 实现直接演化为 P1 步骤 7 调度插件的骨架，S6 仓库留独立 demo 版，P1 用生产版。
6. **验收**：冲突权重-决策曲线图一张；DRA 四条白板要点能脱稿讲（见上，含 v1.35 已 GA 的现状判断，禁止再说”DRA 是未来实验特性”）。

弹药三件套：取舍（NRT annotation fake vs NFD 全家桶、双目标归一化）、坑（KubeSchedulerConfiguration apiVersion/多 profile/schedulerName、RBAC 权限、kind 镜像加载）、数据（权重-决策曲线 + scoringStrategy 对照）。**诚实清单**：全 fake 拓扑；官方 NRT 插件是基线不是对手；没做绑核验证真机性能；DRA 不动手实现，但事实状态（v1.35 GA）与四要点必须跟到能讲、能答迁移期共存问题。

微弹药 M1-M3（按天计的练手小品，不是项目）

定位纪律（违反即失去意义）：

1. **不是项目**：一个目录 + 一份 NOTES.md 即可，不做 README 包装、不写进简历、不占小项目名额——面试官见多了”手写 Docker”的水分项目，但”我写过、代码在这、能讲清原理”就是真弹药；
2. **硬时间盒**：单个 ≤2-3 天，三个全做净工作量 ≤1 周；超时立即走该条目的降级路径，不过夜、不打磨；
3. **窗口纪律**：只允许在寒假、实习空档或两个大项目里程碑之间做；P1 核心 1 冲刺期（步骤 7 前后）禁止开工；

4. **顺序**：M1 所有人必做；M2/M3 按岗位选——平台/调度向 M2 优先，存储/数据流向（你的 P1）M3 优先；有余力才全做。

M1 mini-container（必做，1-2 天）★ 性价比最高

做什么：Go（或 C 直接调 clone）写一个 100-200 行的容器运行器。

- `clone + CLONE_NEWPID/NEWUTS/NEWNS`（有余力加 `NEWNET/NEWUSER`）；
- **cgroup v2**：写 `cgroup.controllers` 体系下的 `cpu.max`、`memory.max`、`memory.swap.max`，把子进程 pid 写入 `cgroup.procs`；
- **rootfs**：busybox 或 distroless 根目录，`pivot_root`（或入门版 chroot）切换，挂载 `/proc`、`/tmp`；
- **stretch**（可选）：`overlayfs lower/upper/merged` 做可写层——正好与 P1 的”`overlayfs` 上 `O_DIRECT` 坑”呼应。

验收：容器内 `ps` 只见自己的进程树；hostname 独立；设 `memory.max=64M` 跑内存分配触发 cgroup OOM 并观察宿主行为；`lsns` 能看到新命名空间。**必须能讲**：容器不是虚拟机（共享宿主内核的含义与攻击面）；cgroup v2 相对 v1 的统一层级变化；`pivot_root` 与 `chroot` 的区别；user namespace 为什么是 rootless 容器的关键；containerd/runC 在 K8s 链路里的位置（kubelet → CRI → containerd → runC）。**降级路径**：`pivot_root` 卡壳就退回 `chroot + mount namespace`，核心目标（namespace/cgroup 手感）不打折。

M2 mini-CNI（平台/网络向，1-2 天）

做什么：写一个能被 CNI 规范真正调用的最简网络插件（Go 可执行文件）。

- 实现 CNI spec 的 `ADD / DEL / CHECK` 命令：从 stdin 读 NetConf JSON、从 `CNI_ARGS` / 环境变量拿 netns 路径；
- **bridge 模式**：建 bridge、给容器 netns 创建 veth pair 一端塞进去、从 /24 地址池（位图或租约文件持久化，防 ADD 重启后泄漏）分配 IP、配接口地址与默认路由、拉起 lo；
- 用 `cnitool` 在两个独立 netns 上验证互通；DEL 释放 IP 并删 veth。

验收：两 netns 互通 ping；重复 ADD 不撞 IP；DEL 后地址回收；`tcpdump` 抓到 bridge 转发。**必须能讲**：K8s “IP-per-pod” 网络模型；CNI 在容器创建链路中的调用时刻（netns 已建、rootfs 之后，由运行时/CRI 调用）；跨节点流量怎么办（VXLAN/路由/BGP——本练习故意不做，要能说清边界）；service iptables/IPVS 与 CNI 为什么是两层。**降级路径**：CNI 调试吃掉半天就停止，改为”读 `containernetworking/plugins` 的 bridge 插件源码 + 用 `ip link/addr/route` 手敲复现全链路”并写笔记。

M3 mini-CSI（存储/数据流向，2-3 天）★ 与 P1 咬合最紧

做什么：写一个最小 CSI 驱动并真的被 kubelet 调用，让 Pod 挂到你提供的卷。

- 实现三个 gRPC service：`Identity`（`GetPluginInfo/Capabilities/Probe`）、`Controller`（最小实现，先不做 provisioning）、`Node`（`NodePublishVolume`：把宿主目录 bind mount 到 kubelet 给的 target path；`NodeUnpublishVolume` 反向卸载）；
- 走 **kubelet plugin registration 机制**：在 `/var/lib/kubelet/plugins_registry/` 暴露注册 socket，与 kubelet 完成握手（这一步本身就是最有价值的弹药）；

- 先做 **static PV + 你的 CSI driver** 的绑定挂载闭环；stretch：实现 `CreateVolume/DeleteVolume`（在宿主建/删目录）+ `StorageClass` 动态供给；
- 在 kind 里以 `DaemonSet`（privileged、挂 `hostPath`）或直接在节点进程两种形态里选简单的先跑通。

验收：PVC 绑定 → 测试 Pod 挂载成功并读写文件 → 删 Pod/PVC 后 `bind mount` 干净卸载（`findmnt` 验证无残留）。**必须能讲：**CSI 三段式 `Attach`（`ControllerPublish`）→ `Stage`（`NodeStageVolume`，块设备格式化/挂全局挂载点）→ `Publish`（`bind mount` 到 Pod）各自解决什么、为什么你的最简实现只需要 `Publish`；`kubelet-registration` 握手与 `socket` 生命周期；ephemeral vs persistent 卷；**Fluid/JuiceFS 为什么需要自己的 CSI driver**（`FUSE` 挂载如何经 CSI 注入 Pod）——这一问直接和 P1 现勘连起来。**降级路径：**动态供给做不完就停在 static PV 闭环（80% 弹药已到手）；kind 里注册不上就在节点上用 `csc` 工具直接调你的 `gRPC` 方法验证。

微弹药与面试域的关系

微弹药	覆盖的面试域	联动
M1	容器运行时、namespace/cgroup、CRI 链路	S6（K8s 下层）、S2 的 overlayfs 坑
M2	CNI、容器网络模型、netns	S6 调度后” Pod 如何落地联网”
M3	CSI、kubelet 卷生命周期、存储编排	P1 现勘（Fluid/JuiceFS CSI）、S4 存储岗叙事

完成顺序与依赖（v3 日历）

现在（使用阶段）

- └ S1 起步（穿插，3–4 周硬顶；P1 早期完成）
- └ P1/P2 双现勘（主战场）

P1 主线期

- └ S6 阶段 A/B（周末预算，约 2–3 周）
- └ P1 步骤 7 ← S6 阶段 C 合流（不另占工期）
- └ S2 随 P1 A 轨推进（不单列）

实习投递（2027.2–4）：P1 核心 1 + S1 + S6（平台岗） / S4（存储岗） + Fluid issue

P1 里程碑之间 / 寒假：M1 必做（1–2 天）；M3 优先（存储/P1 向，2–3 天），M2 平台向

暑期实习后 / 秋招前：S3 填空档（P1 核心 1 已完工为前提）；S4 若改投存储岗则此窗口做

选岗规则：主攻 AI Infra 平台/调度 → S6 + M2；主攻分布式存储/数据流 → S4 + M3。不贪心，正式小项目二选一，S5 不做；M1 无条件做，M2/M3 按岗选，微弹药全部 ≤1 周总量。